

ANDI PROJECT MILESTONES

Milestone-Based Implementation Plan

OVERVIEW

This document outlines the proposed milestone structure for the ANDI version-one build. The goal is to define a clear, controlled implementation path with measurable deliverables, acceptance criteria, scope boundaries, and payment allocation. The plan is intentionally scoped around Gmail, Calendar, LinkedIn-derived relationship data, relationship scoring, model tuning, draft assistance, and a controlled beta-ready product. Additional integrations, expanded production requirements, or broader platform work would be handled separately if approved.

The milestones should generally be completed in sequence, with selected planning and access decisions made early so later work does not require avoidable rework. Each milestone includes the goal, owners, estimated duration, dependencies, deliverables, tasks, acceptance criteria, validation evidence, risks, and out-of-scope boundaries. Estimated timelines and acceptance targets are planning standards, not guarantees of commercial outcomes or unrestricted production readiness.

Executive Note

This milestone plan is meant to align the client and implementation team on what is included in the current ANDI scope, how progress will be evaluated, and where the version-one boundaries are. ANDI may use open-source models where appropriate, but the ANDI product, private integration layer, scoring logic, relationship intelligence, and orchestration code are not being delivered as an open-source product. Generated outreach remains review-first: ANDI may assist with drafts and recommendations, but version one does not send messages automatically without user approval.

Developers: Soham Kinhikar, Bhuvnesh Verma, Jayraj Joshi

Last updated: June 25, 2026

Project Control Definitions

- **Milestone owner** means the accountable person responsible for coordinating implementation, blockers, scope control, and final acceptance evidence. Supporting owners are responsible for implementation support and delivery execution within the assigned milestone.
- **Estimated duration** is a planning estimate, not a guarantee. It assumes normal platform access, timely test data availability, and no major platform changes.
- **Acceptance criteria** define the minimum standard for closing the milestone.
- **Validation evidence** defines what must be shown or documented to prove completion.
- **Out of scope** protects the team from accidental expansion beyond the agreed version-one build.

Ownership Matrix

Soham Kinhikar is the accountable owner for all deliverables/milestones, including Milestone 0, and is responsible for final acceptance evidence, coordination, and scope control. Supporting ownership is assigned as follows:

- **Milestone 0:** Soham Kinhikar.
- **Milestone 1:** Soham Kinhikar, Jayraj Joshi, and Bhuvnesh Verma.
- **Milestone 2:** Soham Kinhikar and Bhuvnesh Verma.
- **Milestone 3:** Soham Kinhikar and Jayraj Joshi.
- **Milestone 4:** Soham Kinhikar, Jayraj Joshi, and Bhuvnesh Verma.
- **Milestone 5:** Soham Kinhikar, Jayraj Joshi, and Bhuvnesh Verma.

Commercial Scope Allocation

The total scoped project value is \$21,500. This allocation is included for milestone-level scope tracking and payment planning. It does not expand the technical scope, create a guarantee of final commercial outcomes, or require work outside the deliverables and acceptance criteria described in this document.

Milestone	Scope allocation basis	Amount
Milestone 0	Customer discovery, implementation planning, architecture alignment, scope definition, and pre-build preparation.	\$1,250
Milestone 1	First part of the shared \$6,000 implementation allocation across Milestones 1–3.	\$2,000
Milestone 2	Second part of the shared \$6,000 implementation allocation across Milestones 1–3.	\$2,000
Milestone 3	Third part of the shared \$6,000 implementation allocation across Milestones 1–3.	\$2,000
Milestone 4	40% of the remaining \$14,250 after Milestone 0 and Milestones 1–3.	\$5,700
Milestone 5	60% of the remaining \$14,250 after Milestone 0 and Milestones 1–3.	\$8,550
Total	Total scoped project value.	\$21,500

Allocation note: Milestones 1–3 are intentionally kept as an approximately even \$6,000 build allocation because they form the foundation of the product. After the \$1,250 Milestone 0 allocation and the \$6,000 foundation allocation, the remaining \$14,250 is split 40% to Milestone 4 and 60% to Milestone 5 because model tuning, production hardening, controlled launch work, monitoring, documentation, and user-facing stabilization carry the largest implementation and validation burden.

Cross-Cutting Privacy and Access Requirements

These requirements apply across the full build and should be considered part of the version-one scope controls.

- Account access permissions for Gmail, Calendar, and LinkedIn-derived data must be documented before real user data is used.
- Credentials, model-service keys, and private configuration values must be stored outside the codebase using a credentials manager or environment-based configuration.
- No generated email, message, or outreach action may be sent automatically without explicit user approval in version one.
- User-specific contacts, messages, calendar events, LinkedIn-derived records, retrieval records, scores, briefs, drafts, and activity records must remain separated by user.
- Data deletion and retention behavior must be documented before onboarding any external beta user.
- The private integration layer and related repositories must be reviewed for leaked credentials, internal URLs, sensitive user data, and implementation details that should remain private.

- Beta logs should avoid storing sensitive message content unless there is a clear product need and access is controlled.

0 MILESTONE 0: CUSTOMER DISCOVERY AND SCOPE ALIGNMENT

Goal: Capture the customer discovery, planning, architecture alignment, scope definition, and pre-build preparation required to make the implementation milestones executable without overcommitting the team beyond the agreed version-one scope.

Primary owner: Soham Kinhikar

Supporting owners: None formally assigned. Jayraj Joshi and Bhuvnesh Verma may support specific technical planning discussions as needed.

Scope allocation: \$1,250 of the \$21,500 total project scope.

Estimated duration: Completed before or during the beginning of implementation; timing depends on customer availability and alignment speed.

Dependencies: Customer conversations, current workflow notes, available sample data, agreed version-one priorities, and confirmation of what will not be included in the first build.

Client or user validation required: Yes. The customer or internal stakeholder should confirm that the planned version-one scope reflects the intended relationship-management workflow.

Scope Notes

Milestone 0 covers discovery and planning work that informs the rest of the build. It should not be described as a production implementation milestone. Its purpose is to reduce ambiguity before engineering work proceeds, especially around the user workflow, data sources, privacy expectations, model behavior, acceptance criteria, and commercial scope. This milestone may include work already completed during customer calls, early product planning, technical architecture review, and milestone drafting.

Deliverables

- Customer discovery summary capturing the target user workflow, relationship-management pain points, expected ANDI use cases, and major version-one priorities.
- Version-one scope summary defining what ANDI is expected to do during the initial build and what is intentionally deferred.
- Technical planning notes covering selected data sources, likely ingestion strategy, model usage approach, private integration layer assumptions, and known access constraints.
- Milestone plan and acceptance criteria aligned to the agreed \$21,500 total project scope.
- Initial risk register covering platform access, LinkedIn limitations, privacy/security expectations, model reliability, cost uncertainty, and beta-readiness assumptions.

Tasks

- ☐ Conduct customer discovery conversations to understand the relationship workflow, current manual process, desired outputs, and pain points.
- ☐ Identify version-one user stories for contact intelligence, signal detection, dossier generation, draft assistance, and morning brief workflows.
- ☐ Clarify available data sources and constraints, including Gmail, Calendar, LinkedIn-derived data, writing samples, and any unavailable systems.
- ☐ Define initial boundaries for user approval, privacy, and no-autonomous-send behavior.
- ☐ Translate discovery findings into implementation milestones, deliverables, acceptance criteria, and out-of-scope boundaries.
- ☐ Confirm commercial scope allocation across Milestone 0 through Milestone 5.

Detailed Acceptance Criteria

- The version-one workflow is documented clearly enough for the team to explain what ANDI will do, what data sources it will use, what outputs it will generate, and what decisions remain user-approved rather than autonomous.
- The milestone plan reflects the agreed \$21,500 scope allocation, including \$1,250 for Milestone 0, \$6,000 split across Milestones 1–3, and the remaining \$14,250 split 40% to Milestone 4 and 60% to Milestone 5.
- Major technical risks are documented before implementation begins, including Gmail/Calendar access, LinkedIn fallback requirements, per-user data handling, model cost uncertainty, and the limits of open-source model usage.
- The team has a shared understanding that ANDI may use open-source models where appropriate but is not itself an open-source product and does not include a public integration layer in version one.
- Scope boundaries are explicit enough to avoid assuming systems or integrations beyond the high-level brief: Gmail, Calendar, LinkedIn-derived data, the private integration layer, model iteration, and the agreed Milestone 5 user-facing/deployment work unless separately approved.

Validation Evidence

- Discovery notes or summary document.
- Updated milestone plan with commercial allocation included.
- Written list of included and excluded version-one functionality.
- Initial risk register or equivalent planning notes.

Risk Factors

- Discovery can create expectations that exceed the implementation budget. Any new feature identified during discovery should be placed in future scope unless it is explicitly added to a milestone.
- Customer needs may change after implementation begins. Material changes should trigger scope review rather than being silently absorbed into existing milestones.
- Early technical assumptions may change once real data is available. The milestone plan should allow implementation details to adjust without expanding promised outcomes.

Out of Scope for Milestone 0

- Production deployment, model tuning, user onboarding, or implementation work that belongs to later milestones.
- Platform availability, model behavior, or external beta readiness beyond the version-one acceptance criteria.
- Finalized UI design or implementation unless separately added to later milestone scope.

1 MILESTONE 1: DATA INGESTION, CLEANING, AND STORAGE

Goal: Build the first reliable data foundation for ANDI by ingesting Gmail, Calendar, and LinkedIn-derived relationship data; normalizing the records; storing them in a per-user database; and indexing content for later retrieval and reasoning.

Primary owner: Soham Kinhikar

Supporting owners: Jayraj Joshi and Bhuvnesh Verma

Scope allocation: \$2,000 of the \$21,500 total project scope. This is one third of the shared \$6,000 allocation across Milestones 1–3.

Estimated duration: 1.5–3 weeks, depending on platform access, account size, and LinkedIn import method.

Dependencies: Gmail account access, Calendar account access, selected database, selected retrieval-indexing and storage approach, and access to the available LinkedIn-derived data source.

Client or user validation required: Yes. A test user must confirm that the ingested contacts and relationship history are directionally accurate.

Scope Notes

This milestone should prioritize reliable ingestion and data quality over advanced intelligence. The AI layer should be limited to extraction, chunking, retrieval indexing, and memory initialization needed to support later scoring and dossier generation. Multi-user architecture should begin here at the data structure level, even if only one user is active during testing.

If direct LinkedIn ingestion is blocked or unstable, version one should support LinkedIn data through CSV export, manual import, or a controlled normalization pass rather than blocking the entire product on live LinkedIn automation.

Deliverables

- Working Gmail ingestion pipeline with account access configuration, thread/message parsing, header normalization, deduplication, and ingestion logs.
- Working Calendar ingestion pipeline with event parsing, attendee extraction, timestamps, recurrence handling where feasible, and ingestion logs.
- LinkedIn normalization/import flow using the available source format, with a fallback path for CSV or export-based ingestion.
- Beta-ready per-user database structure covering users, contacts, email messages, calendar events, LinkedIn records, retrieval records, contact graph edges, scores, briefs, drafts, and activity logs.
- Contact Knowledge Graph data structure and initial graph build using tier 1, tier 2, and tier 3 relationship categories.
- Search-indexing pipeline with documented retrieval approach, chunking strategy, and storage method, and retrieval smoke tests.
- Interruption handling for large ingestion jobs.
- Daily, weekly, or manual ingestion schedule with reporting summary.
- Gmail draft tool-calling foundation that can create and update drafts without sending them automatically.

Tasks

- ☐ Set up Gmail ingestion through the private integration layer, including account access configuration, thread retrieval, pagination, platform-limit handling, and retry behavior.

- ☐ Set up Calendar ingestion through the private integration layer, including event retrieval, attendee extraction, event metadata parsing, and recurrence handling where feasible.
- ☐ Set up LinkedIn ingestion or import path via private integration layer, CSV/export, or controlled browser automation depending on access constraints.
- ☐ Normalize email headers, sender/recipient fields, timestamps, and thread identifiers across all ingested email threads.
- ☐ Define the per-user database structure for users, contacts, messages, calendar events, LinkedIn records, retrieval records, contact graph edges, relationship scores, briefs, drafts, and activity logs.
- ☐ Build the Contact Knowledge Graph with tier 1, tier 2, and tier 3 relationship categories.
- ☐ Implement contact deduplication across Gmail, Calendar, and LinkedIn-derived records.
- ☐ Select and configure the retrieval indexing model, chunking strategy, and search storage method.
- ☐ Index normalized email, calendar, and LinkedIn-derived content and verify retrieval quality with sample queries.
- ☐ Add interruption handling so large ingestion jobs can continue safely or restart without duplicating records.
- ☐ Run LinkedIn data normalization pass to clean lead summaries, profile notes, and messages.
- ☐ Initialize the memory layer using mem0.ai or an equivalent approach for contact-level memory persistence and retrieval.
- ☐ Build ingestion cadence options for daily, weekly, monthly, and manual refresh modes.
- ☐ Build ingestion reports showing processed records, skipped records, duplicates, failures, and data-source coverage.
- ☐ Implement Gmail draft tool-calling foundation for creating and updating drafts without automatic sending.

Acceptance Criteria

- Gmail history for the selected test account is ingested, deduplicated, stored, and indexed for retrieval within the agreed scope of available API permissions.
- Calendar history for the selected test account is ingested, normalized, stored, and linked to known contacts where possible.
- LinkedIn-derived data is imported and normalized using either direct ingestion or the documented fallback path.
- Contact Knowledge Graph is built from email, calendar, and LinkedIn-derived records.
- A large ingestion job can recover from an interruption or restart cleanly without duplicating records.
- Bot-filtered or irrelevant contact list is reviewed against a manually labeled sample, with a target false-positive rate below 2 percent. If the target is not reached, the remaining failure cases must be documented with a remediation plan.
- Database structure supports per-user storage for all major entities and can be recreated cleanly from data setup.
- Ingestion pipeline produces a readable summary report after each run.
- No email sending behavior exists without explicit user approval.

Validation Evidence Required

- Ingestion report showing record counts by source: Gmail messages, Gmail threads, Calendar events, LinkedIn records, contacts, graph edges, and retrieval records.
- Database data-structure update files and successful clean-database data-structure update test.
- Sample retrieval test showing that context search returns relevant relationship context.
- Interruption-handling test using a simulated ingestion failure.
- Manual review sheet for bot filtering and contact deduplication.

Risk Factors

- Large ingestion recovery can add complexity. If unstable, the team may use documented manual restarts temporarily and revisit after the first reliable ingestion pass.
- LinkedIn data may have format drift or access restrictions. Validate source format early and maintain a CSV/export fallback.
- Data retention policies and privacy concerns are significant because Gmail, Calendar, and relationship data are sensitive.
- Large Gmail histories may increase ingestion time and model and retrieval-indexing cost. Track cost and runtime early.

Out of Scope for Milestone 1

- Fully autonomous outreach.
- Polished production frontend.
- Final scoring model calibration.
- Multi-user external launch beyond data structure readiness and local/internal testing.

2 MILESTONE 2: SCORING AND NOISE MODEL

Goal: Convert ingested relationship data into usable signal by ranking contacts, identifying meaningful relationship events, filtering noise, and producing an accurate morning brief.

Primary owner: Soham Kinhikar

Supporting owners: Bhuvnesh Verma

Scope allocation: \$2,000 of the \$21,500 total project scope. This is one third of the shared \$6,000 allocation across Milestones 1–3.

Estimated duration: 2–3 weeks after Milestone 1 is complete.

Dependencies: Completed ingestion pipeline, stable Contact Knowledge Graph, labeled sample contacts, enough Gmail/Calendar/LinkedIn-derived data for evaluation, and initial cost tracking from ingestion.

Client or user validation required: Yes. The primary user must review contact rankings, false positives, and the morning brief quality.

Deliverables

- Network scoring system using Gmail, LinkedIn-derived data, Calendar activity, and ANDI-specific subscores.
- Signal taxonomy defining what counts as a useful signal, weak signal, duplicate signal, irrelevant signal, and false positive.
- Triage rules covering promote, skip, and hold decisions.
- Four-layer evaluation harness covering extraction accuracy, scoring precision, triage recall, and brief relevance.
- Ranked morning brief generator with signal explanations and references back to source records where possible.
- Calibration report using at least 100 labeled contacts and top-50 contact review.
- Cost-per-contact and cost-per-brief measurement report.

Tasks

- ☐ Define scoring inputs and subscores from Gmail, Calendar, LinkedIn-derived data, and ANDI-specific activity.
- ☐ Implement network scoring using recency, frequency, relationship strength, explicit user importance, response patterns, meeting history, and relevant keywords.
- ☐ Build triage system with 7 envelope rules covering promote, skip, and hold behavior.
- ☐ Define signal taxonomy for calendar events, Gmail threads, LinkedIn-derived updates, keywords, and relationship-change events.
- ☐ Build signal detection for calendar events, Gmail threads, relationship keywords, meeting cadence changes, unanswered messages, and upcoming follow-up opportunities.
- ☐ Build scoring system to prioritize signals by urgency, relationship value, source confidence, freshness, and actionability.
- ☐ Implement phase 3 triage pipeline connecting classifier, routing, and brief-generation logic.
- ☐ Run triage rule iteration against real data from Milestone 1.
- ☐ Build 4-layer eval harness covering extraction accuracy, scoring precision, triage recall, and brief relevance.
- ☐ Implement adaptive tier boundaries that adjust when contact volume changes by more than 20 percent.
- ☐ Refine signal scoring to produce a ranked morning brief.
- ☐ Calibrate network scores by reviewing the top 50 contacts and spot-checking low-ranked contacts.

- ☐ Implement cost-per-contact and cost-per-brief measurement using model usage and model call tracking.
- ☐ Collect ground-truth labels from at least 100 contacts for calibration.
- ☐ Document scoring formula, weighting assumptions, and known limitations.

Acceptance Criteria

- Triage rules are evaluated against a 100-contact manual label set, with a target agreement rate above 90 percent. If the target is not reached, the milestone can close only with documented failure categories, examples, and next-step tuning recommendations.
- Top-20 network-score ranking is reviewed against expected relationship strength, with a target of at least 80 percent reviewer agreement. Material mismatches must be documented with likely causes such as missing data, stale data, weighting error, or ambiguous relationship history.
- Morning brief is evaluated during a defined test window, with a target of fewer than 3 false-positive signals per day. If the target is missed, false positives must be categorized by source and triage rule before launch use.
- Four-layer eval harness runs on every build or evaluation run and produces saved output.
- Cost per contact and cost per brief are measured, documented, and associated with model/tool usage.
- Tier-boundary adjustment logic is tested against at least two contact-volume scenarios, including a greater-than-20-percent volume change. The output should preserve a reasonable tier distribution, or the exceptions must be documented before use.
- Every promoted signal includes a short explanation of why it was promoted.

Validation Evidence Required

- 100-contact labeled calibration set with promote, skip, hold, or tier labels.
- Top-50 contact review sheet showing expected versus actual ranking.
- Evaluation harness output for extraction accuracy, scoring precision, triage recall, and brief relevance.
- Sample morning briefs with false positives marked.
- Cost report showing cost per contact, cost per brief, and most expensive operations.

Risk Factors

- Scoring calibration is meaningless without full-enough data from Milestone 1. This milestone should not begin until ingestion quality is acceptable.
- Cost per contact may exceed what is viable for the unit economics. Identify the most expensive operations early and plan optimization in Milestone 4.
- Adaptive tier boundaries trained on a single user may overfit. Validate logic against at least two distinct contact volumes before finalizing.
- A model can appear accurate while still missing high-value relationship context. Manual user review is required.

Out of Scope for Milestone 2

- Final polished UI.
- Fully polished dossier generation.
- Voice-matched email drafting beyond basic signal-to-draft experimentation.
- External multi-user launch.

3 MILESTONE 3: PRIVATE INTEGRATION LAYER

Goal: Build a private integration layer that supports ANDI’s internal tool calls, Gmail and Calendar connectors, authentication, documentation, and test coverage. This milestone does not include publishing a public integration layer or open-sourcing ANDI product code.

Primary owner: Soham Kinhikar

Supporting owners: Jayraj Joshi

Scope allocation: \$2,000 of the \$21,500 total project scope. This is one third of the shared \$6,000 allocation across Milestones 1–3.

Estimated duration: 1.5–2.5 weeks, partially parallelizable with Milestone 2 after the internal interface design is stable.

Dependencies: Stable internal ingestion interfaces, authentication decision, test accounts, packaging decision, private repository access, and security review checklist.

Client or user validation required: No for initial internal build. User validation is only required if the integration layer affects visible Gmail, Calendar, draft, or brief behavior.

Scope Notes

The integration layer should expose a clean private interface for the ANDI application to call supported tools and data sources. It may follow standard integration-layer patterns, but it is not intended to be a public integration layer, public package, or open-source repository. The product may use open-source models in selected model workflows, but the ANDI application, scoring logic, relationship intelligence, private data structures, and internal orchestration code remain private.

This milestone should avoid overbuilding a developer platform. The goal is reliable internal integration, testable tool calls, secure credentials, and maintainable boundaries between ingestion, scoring, model generation, and user-facing workflows.

Deliverables

- Private integration-layer specification covering internal tool names, request format, response format, error format, retry behavior, and change-management notes.
- Private repository or internal package with documented setup for the ANDI team.
- Gmail connector callable through the internal integration layer after documented account access configuration.
- Calendar connector callable through the internal integration layer after documented account access configuration.
- Internal authentication layer using private keys, service credentials, or another agreed mechanism suitable for the private beta environment.
- Internal README, setup guide, internal tool reference, and example environment file.
- Integration test suite covering connector calls, error paths, authentication failures, and parity with the internal ingestion endpoints where applicable.
- Documented test workflow for linting, formatting, unit tests, and integration tests in the private repository.
- Security review checklist completed before using the integration layer with real user data.

Tasks

- ☐ Define the internal integration-layer tool interface, including naming conventions, required inputs, optional inputs, output format, error format, timeout behavior, and retry rules.
- ☐ Document which tool calls are required for version one: Gmail ingestion, Calendar ingestion, contact lookup, draft creation/update, retrieval, and status reporting.

- ☐ Define the boundary between the integration layer and private ANDI product logic so scoring, ranking, dossier generation, voice profiles, and user data remain inside the private application layer.
- ☐ Decide the private repository structure, package/module layout, runtime environment, dependency management approach, and release process for internal use.
- ☐ Build the authentication layer with API-key, internal credential, or internal secret-based access control.
- ☐ Add Gmail connector support for configured accounts, including pagination, platform-limit handling, thread/message retrieval, attachment metadata where needed, and safe failure behavior.
- ☐ Add Calendar connector support for configured accounts, including event retrieval, attendee extraction, recurrence handling where feasible, and safe failure behavior.
- ☐ Add configuration templates documenting required environment variables, account access credentials, model service credentials where relevant, and local/staging separation.
- ☐ Write internal documentation: README, setup guide, tool reference, example calls, common errors, and troubleshooting notes.
- ☐ Write integration tests for successful tool calls, invalid credentials, missing configuration, platform-limit responses, empty accounts, malformed records, and partial failures.
- ☐ Set up a documented test command that runs formatting checks, unit tests, and integration tests before changes are merged.
- ☐ Conduct security review before using real user data to check for leaked credentials, internal URLs, private data structures, sensitive user data, and unsafe logging.
- ☐ Confirm that no public release, public package, public repository, or external developer-support obligation is included in version one.

Acceptance Criteria

- **Private interface readiness:** The internal integration-layer interface is documented well enough for an ANDI developer to add or call a supported tool without reading unrelated product code. The documentation must include tool names, inputs, outputs, error codes, expected retries, and authentication requirements.
- **No public/open-source product claim:** The milestone documentation, repository metadata, README, and release notes must not describe ANDI as an open-source product or describe this work as a public integration layer. Any reference to open source must be limited to open-source models or libraries used as dependencies.
- **Connector functionality:** Gmail and Calendar connectors must successfully run in a configured development or staging environment using approved test credentials. The test output must show at least one successful Gmail retrieval path and one successful Calendar retrieval path, unless a connector is blocked by third-party access limits that are documented as a dependency issue.
- **Authentication coverage:** Internal authentication must be present, documented, and tested for success and failure cases. Invalid or missing credentials must fail safely without exposing credentials or sensitive user data.
- **Configuration clarity:** Required environment variables, account access credentials, local/staging separation, and setup steps must be documented. A developer with the correct private credentials should be able to configure the integration layer without guessing hidden setup steps.
- **Test coverage:** Integration tests must cover successful connector calls, invalid credentials, missing configuration, empty results, malformed records, and partial failures. Any tests that require live credentials must be clearly marked and separated from offline tests.
- **Security review:** Before the integration layer is used with real user data, the security checklist must confirm that credentials are not committed, logs do not expose sensitive raw content by default, internal URLs are not unnecessarily exposed, and user data is not included in fixtures or documentation.
- **Private boundary:** No private ANDI scoring logic, relationship-ranking logic, dossier-generation

logic, voice-profile data, user data, credentials, or internal product strategy is exposed outside the private repository or internal deployment boundary.

- **Scope control:** There is no requirement in this milestone to publish a public package, support third-party developers, maintain a public repository, or provide a hosted external integration-layer service.

Validation Evidence Required

- Internal interface specification or tool-reference document.
- Private setup guide and example configuration file with credentials redacted.
- Test output showing Gmail and Calendar connector calls in development or staging.
- Authentication success/failure test output.
- Integration test report or local test log.
- Security review checklist confirming no committed credentials, no unsafe logging, and no public/open-source release claims.
- Boundary review note confirming that scoring, ranking, dossier generation, voice profile, and user data remain private.

Risk Factors

- Expanding the integration layer beyond the private ANDI workflow would create unnecessary scope and maintenance burden. Keep the version-one layer private and narrow.
- Gmail and Calendar behavior may vary by account permissions, account access permission approval, platform limits, and account size. Failures should be documented rather than treated as guaranteed platform support.
- Security issues are easy to introduce when working with account access credentials and user data. Review logging, fixtures, screenshots, and config files before real user testing.
- Internal interface changes can break ingestion, scoring, or draft workflows. Version the interface and maintain integration tests before changing request or response data structures.

Out of Scope for Milestone 3

- Public integration release.
- Open-sourcing ANDI product code, scoring logic, ranking logic, dossier-generation logic, or internal orchestration code.
- Public repository, public package distribution, public release process, or external developer-support process.
- Supporting data sources beyond Gmail, Calendar, and LinkedIn-derived data unless separately approved.
- A full hosted SaaS deployment of the integration layer as a standalone product.

4 MILESTONE 4: MODEL ITERATION AND TUNING

Goal: Improve ANDI’s generated relationship intelligence by tuning extraction, dossier generation, patching, drafting, source-support review, voice match, reliability, regression testing, and cost performance.

Primary owner: Soham Kinhikar

Supporting owners: Jayraj Joshi and Bhuvnesh Verma

Scope allocation: \$5,700 of the \$21,500 total project scope. This represents 40% of the remaining \$14,250 after Milestone 0 and Milestones 1–3.

Estimated duration: 2–4 weeks, depending on evaluation volume, writing-sample availability, and model/provider experimentation.

Dependencies: Milestone 2 evaluation harness, representative dossiers, source references, draft samples, cost tracking, prompt tracking process, and enough writing samples to tune the voice profile.

Client or user validation required: Yes. Voice match, dossier usefulness, source-support quality, and relationship sensitivity require human review.

Scope Notes

Milestone 4 is the quality, reliability, and cost-control phase. The purpose is not simply to “make the model better,” but to make each generated output measurable, reviewable, source-grounded, and safe to use. Every prompt change should be treated like a product change: it needs a baseline, a test set, a measured result, and a regression check.

This milestone should preserve the version-one safety boundary: ANDI may draft recommendations, summaries, talking points, and messages, but it should not send outreach automatically. Any user-facing message generated in this milestone must remain in draft or approval-required form.

Deliverables

- Dossier generation pipeline supporting structured and narrative outputs using approved internal formats.
- Field-level dossier specification covering required fields, optional fields, source-reference rules, missing-evidence behavior, and confidence labels.
- Patch system that can revise generated dossiers, briefs, and drafts for writing style, risk flags, relationship context, accuracy, and talking points without rewriting unrelated sections.
- Draft-generation pipeline using a user-specific voice profile while preserving explicit user approval before sending.
- Source-support review system that checks whether generated claims are supported by source emails, calendar events, LinkedIn-derived records, contact graph records, or user-provided notes.
- Model and prompt iteration log showing baseline metrics, prompt versions, model versions, cost impact, quality impact, and regression results.
- Voice-profile package containing writing samples, extracted style features, prohibited patterns, preferred phrasing, and a documented voice-match rubric.
- Twenty-draft human review set with ratings, comments, failure modes, and final voice-match score.
- Cost-optimization plan identifying which operations stay on premium models, which can move to cheaper hosted models, and which can be tested on open-source models.
- Benchmark report for at least one extraction task on an appropriate open-source model candidate. This benchmark is exploratory and does not commit the product to using that model in production.
- Prompt/token optimization report comparing baseline usage against optimized usage with quality preserved.

- Regression test suite covering extraction quality, source grounding, unsupported-claim rate, draft safety, model cost, and output format validity.

Detailed Task Breakdown

Task 4.1: Build dossier generation pipeline with structured and AI-assisted output formats

- ☐ Define the purpose of a dossier: what decision it helps the user make, what relationship context it summarizes, and what action it should support.
- ☐ Define the structured dossier format with explicit fields for contact identity, relationship tier, relationship summary, recent interactions, open loops, potential opportunities, recommended next action, risks, and source references.
- ☐ Define the v5 AI-assisted dossier format that can produce a more narrative, strategy-oriented summary while still mapping back to the structured fields.
- ☐ Implement structured output validation so every dossier response is machine-checkable before it is shown to a user.
- ☐ Implement fallback behavior for missing evidence, including “unknown,” “not enough evidence,” or “needs user confirmation” rather than fabricated claims.
- ☐ Implement contact-level context lookup from Gmail, Calendar, LinkedIn-derived records, retrieval records, contact graph edges, and stored memory.
- ☐ Add dossier version metadata, including prompt version, model version, generation timestamp, contact ID, user ID, source IDs, and cost estimate.
- ☐ Create at least 10 representative dossier examples across high-touch contacts, weak ties, dormant contacts, recent meetings, and noisy contacts.

Task 4.2: Define required fields, optional fields, citation requirements, and missing-evidence behavior

- ☐ Mark each dossier field as required, optional, derived, or user-confirmed.
- ☐ Define which fields require direct source references, such as recent interactions, meeting references, open loops, and specific claims about relationship history.
- ☐ Define which fields can be model-inferred but must carry a confidence label, such as relationship warmth or suggested next action.
- ☐ Define which fields must never be invented, including employer, title, personal details, commitments, promises, meetings, deadlines, and sensitive relationship information.
- ☐ Add a “citation missing” state that blocks or flags any generated claim that cannot be traced to a stored source record.
- ☐ Add clear behavior for stale data, including when old interactions should be marked as historical rather than current.
- ☐ Document examples of acceptable and unacceptable generated claims.

Task 4.3: Build patch system for writing style, risk flags, relationship context, and talking points

- ☐ Define patch categories: style patch, factual patch, risk patch, talking-point patch, tone patch, and relationship-context patch.
- ☐ Implement a patch input format that specifies the target output, target section, requested change, reason for change, and whether source references must be rechecked.
- ☐ Ensure patches can update one section without regenerating the full dossier or draft unless the change affects global context.
- ☐ Add risk-flag patches for overconfident claims, sensitive personal references, unsupported assumptions, overly sales-oriented language, and missing user approval.

- ☐ Add talking-point patches that can convert relationship context into concise meeting prep bullets.
- ☐ Add writing-style patches that adjust tone, directness, length, specificity, and formality while preserving facts and source references.
- ☐ Log every patch with before/after text, patch type, source reason, prompt version, and model version.
- ☐ Add regression checks to make sure patching does not remove source references, change factual meaning, or introduce unsupported claims.

Task 4.4: Build draft generation with voice profile integration

- ☐ Define supported draft types for version one, such as follow-up email, check-in note, intro request, meeting recap, thank-you note, and soft reactivation message.
- ☐ Build a voice profile from approved writing samples, including typical sentence length, greeting style, directness, warmth, sign-off style, use of context, and level of formality.
- ☐ Separate factual content from style instructions so the voice layer cannot create unsupported facts.
- ☐ Add draft constraints requiring the model to avoid overpromising, pressure tactics, sensitive references, and claims not present in the source context.
- ☐ Require every generated message to remain a draft and include an explicit approval boundary before sending.
- ☐ Add source-grounded context cards beside or inside the draft-generation process so the user can see why the draft was recommended.
- ☐ Generate a 20-draft review set across different relationship types and outreach scenarios.
- ☐ Store draft metadata, including source contact, source signals, voice-profile version, prompt version, cost, and approval status.

Task 4.5: Implement model synthesis with source-support review

- ☐ Define citation objects for emails, calendar events, LinkedIn-derived records, contact graph edges, and user notes.
- ☐ Require generated factual claims to reference one or more source objects where feasible.
- ☐ Implement a claim-checking pass that compares generated claims against retrieved source snippets or structured records.
- ☐ Classify each checked claim as supported, partially supported, unsupported, stale, or ambiguous.
- ☐ Block or flag unsupported claims in dossiers, briefs, and drafts before user review.
- ☐ Add source-reference coverage metrics showing what percentage of factual claims have direct support.
- ☐ Add unsupported claims-rate tracking by output type: dossier, brief, draft, and talking points.
- ☐ Create a manual review workflow for a reviewer to inspect unsupported or ambiguous claims.

Task 4.6: Improve extraction quality using the Milestone 2 eval harness

- ☐ Identify the weakest extraction categories from Milestone 2, such as names, organizations, commitments, dates, follow-up requests, relationship events, or meeting intent.
- ☐ Create an error taxonomy for extraction failures: missed entity, wrong entity, wrong date, wrong relationship, duplicate signal, unsupported inference, stale signal, or irrelevant signal.
- ☐ Update extraction prompts, data structures, or routing logic based on the error taxonomy.
- ☐ Rerun the evaluation harness after each meaningful change and compare against the Milestone 2 baseline.
- ☐ Track precision, recall, F1 where applicable, false-positive rate, false-negative rate, and cost per extraction.
- ☐ Save evaluation artifacts so regressions can be traced to specific prompt or model changes.

Task 4.7: Iterate dossier prompts to reduce unsupported claims and improve output structure

- ☐ Establish a baseline unsupported-claim rate, structure-validity rate, source-reference-coverage rate, and reviewer usefulness score.
- ☐ Create prompt variants targeting one issue at a time, such as shorter context windows, stricter citation rules, better missing-data handling, or improved action recommendations.
- ☐ Run each prompt variant against the same evaluation set before selecting a winner.
- ☐ Reject prompt variants that improve style but increase unsupported claims or source-support failures.
- ☐ Add explicit instruction patterns for uncertainty, missing data, stale data, and weak evidence.
- ☐ Compare structured v4 output against AI-assisted v5 output for accuracy, usefulness, and cost.
- ☐ Document the final selected prompt and the reason it was selected.

Task 4.8: Run draft quality iteration with voice-match scoring

- ☐ Define the 20-draft evaluation sample across warm contacts, cold contacts, investors, partners, mentors, dormant contacts, and recent meeting follow-ups.
- ☐ Score each draft using the documented voice-match rubric.
- ☐ Track separate scores for voice accuracy, factual accuracy, usefulness, relationship sensitivity, brevity, clarity, and likelihood of user approval.
- ☐ Identify recurring draft problems, such as sounding too formal, too pushy, too generic, too verbose, or too disconnected from context.
- ☐ Modify voice prompts and examples based on the review results.
- ☐ Rerun the 20-draft review after major voice-profile changes.
- ☐ Maintain a rejected-pattern list so the system avoids phrases or behaviors the user dislikes.

Task 4.9: Tune voice profile with additional writing samples

- ☐ Collect approved writing samples from emails, text-style messages, professional notes, and prior drafts.
- ☐ Separate samples by context, such as formal client communication, casual professional follow-up, investor communication, and internal team communication.
- ☐ Extract reusable style features, including greeting patterns, sentence length, directness, level of warmth, use of humor, and typical sign-offs.
- ☐ Define prohibited style patterns, including language that sounds too corporate, overly apologetic, overly aggressive, generic, or fake.
- ☐ Create a voice-profile version history so changes can be rolled back.
- ☐ Test the voice profile against the 20-draft sample and compare scores before and after tuning.

Task 4.10: Create and use a voice-match rubric

- ☐ Define a 1–10 scoring scale with written descriptions for each score band.
- ☐ Score drafts across at least five categories: tone, structure, specificity, relationship sensitivity, and naturalness.
- ☐ Add a factuality gate so a draft with unsupported facts cannot receive a passing score even if it sounds like the user.
- ☐ Add a safety gate so a draft that pressures, manipulates, or overpromises cannot receive a passing score.
- ☐ Require reviewer comments for any score below 7 so prompt changes have actionable feedback.
- ☐ Document examples of passing and failing drafts.

Task 4.11: Document cost optimization plan

- ☐ Break total cost into ingestion, retrieval indexing, extraction, scoring, dossier generation, brief generation, draft generation, source-support review, and patching.

- ☐ Identify the top three cost drivers using actual token/model call data.
- ☐ Classify operations into premium-model required, cheaper-hosted-model eligible, open-source-model candidate, cacheable, or avoidable.
- ☐ Define caching rules for repeated context lookup, repeated dossier generation, and unchanged contact histories.
- ☐ Define when to use smaller models for extraction or classification and when to escalate to larger models for synthesis.
- ☐ Estimate cost per contact, cost per brief, cost per dossier, cost per draft, and projected cost per active user.
- ☐ Document cost-quality tradeoffs and recommend the default model routing plan for beta launch.

Task 4.12: Benchmark at least one extraction task on an open-source model

- ☐ Select one extraction task with a clear expected output, such as commitment extraction, date extraction, contact-detail extraction, or follow-up signal extraction.
- ☐ Choose at least one open-source model candidate for exploratory benchmarking only.
- ☐ Run the same evaluation sample through the current hosted-model baseline and the open-source candidate.
- ☐ Compare accuracy, latency, cost, structured output adherence, citation compatibility, and failure modes.
- ☐ Record whether the open-source model candidate appears suitable for user-facing use, offline/batch use only, or not acceptable based on the measured sample.
- ☐ Document any infrastructure implications, such as hosting cost, GPU need, latency, quantization, or privacy advantages.

Task 4.13: Optimize prompts to reduce token usage without quality loss

- ☐ Establish token-usage baselines for extraction, dossiers, briefs, drafts, source-support review, and patches.
- ☐ Remove redundant instructions, repeated context, overly large examples, and unnecessary message history from prompts.
- ☐ Replace long natural-language instructions with compact structured data structures where possible.
- ☐ Add retrieval limits so only relevant source context is passed into synthesis.
- ☐ Add summarization or caching for long contact histories that do not change frequently.
- ☐ Compare optimized prompts against the baseline evaluation set.
- ☐ Confirm that token reduction does not reduce extraction accuracy, source-reference coverage, voice-match score, or dossier usefulness.

Task 4.14: Add regression tests for prompt and model changes

- ☐ Build a fixed regression dataset of contacts, source snippets, calendar events, expected extractions, expected signal classifications, and expected output constraints.
- ☐ Add structured-output-validity tests for dossiers, briefs, drafts, and patch outputs.
- ☐ Add factuality tests for citation-supported claims.
- ☐ Add draft-safety tests to ensure outputs remain approval-required and do not claim messages were sent.
- ☐ Add cost regression tests that flag major token increases.
- ☐ Add minimum-quality thresholds so prompt changes cannot be merged if they reduce key metrics below acceptance targets.
- ☐ Store test results with prompt version and model version.

Extremely Detailed Acceptance Criteria

- **Dossier data structure readiness:** v4 structured dossier and v5 AI-assisted dossier formats are fully documented, implemented, and data structure-validated. A generated dossier must include contact identity, relationship tier, relationship summary, recent interactions, open loops, opportunities, recommended next action, risks, confidence labels, source IDs, prompt version, model version, generation timestamp, and estimated generation cost.
- **Dossier field completeness:** On the evaluation set, at least 95% of generated dossiers must include all required fields. Any missing required field must be replaced with an explicit “unknown” or “not enough evidence” state rather than fabricated content.
- **Source grounding:** At least 90% of factual claims in dossiers and briefs must be linked to a source object or explicitly marked as an inference. Unsupported factual claims must be flagged before user review.
- **Unsupported claims control:** Dossier unsupported-claim rate must be below 5% on the evaluation set. A unsupported claims is any generated factual claim that is contradicted by the source record, not present in the source record, or presented as certain when the evidence is ambiguous.
- **Extraction improvement:** Extraction accuracy must improve by more than 10% over the Milestone 2 baseline on the same evaluation harness. The comparison must use the same or a clearly documented equivalent test set, and the report must show baseline score, tuned score, sample size, and major error categories.
- **Regression safety:** No accepted prompt or model change may reduce extraction accuracy, source-reference coverage, data structure validity, or draft-safety pass rate below the agreed thresholds. Any accepted tradeoff must be explicitly documented with rationale.
- **Patch correctness:** The patch system must successfully apply style, factual, risk, talking-point, and relationship-context patches to sample outputs without removing required source references, changing unrelated sections, or introducing unsupported facts.
- **Draft generation safety:** All generated outreach must remain in draft or approval-required form. No workflow in this milestone may send email, calendar invites, LinkedIn messages, or other outreach automatically.
- **Draft voice match:** Draft voice match must average at least 7 out of 10 across a 20-draft review set using the documented rubric. No individual draft may pass if it contains unsupported factual claims, unsafe pressure language, or an overpromise.
- **Voice rubric completeness:** The voice rubric must define scoring bands for tone, structure, specificity, relationship sensitivity, naturalness, factuality, and safety. Drafts scoring below 7 must include reviewer notes explaining the failure mode.
- **Cost plan completeness:** The cost-optimization plan must identify actual cost per extraction, dossier, brief, draft, patch, contact, and active user estimate. It must also identify the top three cost drivers and recommend model routing for beta launch.
- **Token usage reduction:** Total token usage for the selected evaluation workflow must be reduced by at least 15% from the Milestone 2 baseline without reducing extraction accuracy, source-reference coverage, dossier usefulness, or voice-match score. A 20% or greater reduction is the stretch target.
- **Open-source model benchmark:** At least one extraction task must be benchmarked on an open-source model candidate against the hosted-model baseline. The report must include accuracy, latency, cost estimate, structured output adherence, failure modes, and a recommendation on whether the model is suitable for user-facing use, batch-only use, or not acceptable for the tested task. This does not imply that ANDI itself is open source.
- **Evaluation artifact retention:** All evaluation outputs, review sheets, prompt versions, model versions, model-usage reports, and benchmark results must be saved in a reproducible location so later regressions can be traced.

- **User review readiness:** The milestone cannot be closed until a human reviewer has inspected representative dossiers, drafts, source references, and risk flags and confirmed that the outputs are useful enough for beta-level use.

Validation Evidence Required

- Milestone 2 baseline versus Milestone 4 tuned evaluation report.
- Dossier format documentation for the approved internal outputs.
- At least 10 representative dossiers with source-support results.
- Source-support review report showing supported, partially supported, unsupported, stale, and ambiguous claims.
- 20-draft voice-match review sheet with reviewer comments and final score averages.
- Patch-system test results showing successful patch types and failure cases.
- Prompt and model iteration log with prompt version, model version, date, reason for change, cost impact, quality impact, and regression result.
- Open-source model benchmark report.
- Model-usage report showing baseline usage, optimized usage, percentage reduction, and quality-preservation checks.
- Cost-optimization plan with recommended model routing for beta launch.

Risk Factors

- Model iteration without a cost plan can burn budget quickly. Cost tracking must happen before large-scale prompt experiments.
- Voice match is subjective. The rubric must be defined before ratings are collected, or the 7/10 target will not be meaningful.
- Open-source models may not meet the quality bar for extraction or source-grounded tasks. Benchmark with a realistic sample before using them in any user-facing workflow.
- Over-optimization for one user's writing style can reduce generalizability. Keep user-specific voice behavior separate from reusable system logic.
- Source-support review may increase latency and cost. The team should measure whether verification runs inline, async, or only for high-risk outputs.
- Aggressive model-usage reduction can accidentally remove important relationship context. Optimization must be evaluated against quality, not just cost.

Out of Scope for Milestone 4

- Fully autonomous sending or outreach execution.
- Controlled beta launch readiness.
- Model fine-tuning unless explicitly approved after benchmarking and cost review.
- Replacing the whole application architecture solely to support a model experiment.

5 MILESTONE 5: FINAL PRODUCT

Goal: Move ANDI from prototype toward a controlled beta-ready product by finalizing the core data structure, separating each user’s data, deploying the agreed version-one environment, adding monitoring, documenting deletion behavior, preparing user-facing guidance, and validating the minimum beta workflow.

Primary owner: Soham Kinhikar

Supporting owners: Jayraj Joshi and Bhuvnesh Verma

Scope allocation: \$8,550 of the \$21,500 total project scope. This represents 60% of the remaining \$14,250 after Milestone 0 and the Milestones 1–3 foundation allocation.

Estimated duration: 3–6 weeks, depending on hosting decisions, product-surface decision, access stability, and model-cost findings from Milestone 4.

Dependencies: Milestones 1–4 complete or materially stable, validated cost model, hosting decision, agreed user-facing surface, and approval to use real user data for beta testing.

Client or user validation required: Yes. A client or approved beta user should be able to review the minimum ANDI workflow and confirm whether the outputs are useful enough for the agreed version-one beta.

Scope Notes

Milestone 5 is focused on making the agreed ANDI version-one workflow usable in a controlled beta setting. It does not promise a full-scale SaaS rollout, a polished consumer-grade application, autonomous outreach, or additional integrations beyond the approved version-one data sources. The main objective is to make the existing ANDI workflow reliable enough to demonstrate value, collect feedback, and identify what should be improved after v1.

The user-facing surface may be a lightweight web interface, internal dashboard, or integration-style surface, depending on what is approved before this milestone begins. The acceptance standard is beta usability, not full-scale commercial launch readiness.

Deliverables

- Finalized beta data structure and query cleanup for the agreed version-one entities: users, contacts, messages, calendar events, LinkedIn-derived records, retrieval records, relationship scores, briefs, dossiers, drafts, and activity records.
- Per-user data separation for contacts, source records, generated outputs, preferences, and account configuration.
- Controlled hosted beta environment with credentials stored outside the codebase and a documented deployment path.
- Monitoring view or reporting output covering cost per user, token usage, cost per workflow, latency, error rate, ingestion status, and model-call issues.
- Data deletion and account-disconnect flow with documented behavior and tested verification steps.
- Internal operations runbook for common support and recovery scenarios.
- User-facing onboarding guide explaining setup, supported data sources, known limitations, review-before-send behavior, and feedback process.
- Minimum user-facing product surface for reviewing contact ranking, morning brief or equivalent summary, dossier output, and draft/recommendation output.
- Beta-readiness report summarizing what shipped, what was tested, known limitations, cost profile, unresolved risks, and recommended next steps.

Tasks

Task 5.1: Finalize beta data structure and clean up product queries

- ☐ Review the records created during prototype development and identify temporary names, unused fields, duplicated records, and inconsistent query paths.
- ☐ Finalize naming and storage conventions for users, contacts, messages, calendar events, LinkedIn-derived records, retrieval records, scores, briefs, dossiers, drafts, memory records, and activity records.
- ☐ Update the application queries used for ingestion, scoring, dossier generation, draft generation, deletion, and reporting so they use the finalized beta data structure.
- ☐ Test the finalized data structure from a clean setup and from a staging/prototype copy where feasible.
- ☐ Document any temporary fields or workarounds that remain, including the owner and reason for keeping them.

Task 5.2: Implement per-user data separation and configuration

- ☐ Ensure each user's contacts, messages, calendar events, LinkedIn-derived records, retrieval records, scores, briefs, dossiers, drafts, and preferences remain separated.
- ☐ Add checks so one user's generated outputs cannot accidentally use another user's source data.
- ☐ Test the case where two users share the same contact name, contact email, company, or meeting attendee without merging private relationship history.
- ☐ Store per-user settings for account access, timezone, brief preferences, writing style, and enabled features where required for v1.
- ☐ Document the expected behavior for single-user testing, internal testing, and approved beta-user testing.

Task 5.3: Deploy the controlled beta environment

- ☐ Confirm the hosting decision and required services for the application, database, retrieval storage, background jobs, and model calls.
- ☐ Configure the beta environment so it can run outside a developer laptop.
- ☐ Store credentials, account-access values, model-service keys, and private configuration outside the codebase.
- ☐ Add basic health checks or documented manual checks for the application, database, retrieval storage, ingestion jobs, and model service connectivity.
- ☐ Document the deployment steps, rollback steps, and required environment variables for the team.

Task 5.4: Add monitoring and cost visibility

- ☐ Track cost per user, cost per brief, cost per dossier, cost per draft, and token usage by workflow.
- ☐ Track latency for ingestion, retrieval, scoring, dossier generation, brief generation, and draft generation.
- ☐ Track common failure categories such as ingestion failures, account-access failures, model-call failures, retrieval failures, and output-format failures.
- ☐ Create a simple dashboard, report, or exported log summary that the team can review during beta.
- ☐ Define who reviews beta health metrics and how often they should be reviewed during the beta period.

Task 5.5: Implement data deletion and account-disconnect behavior

- ☐ Define what user data should be deleted or disconnected, including account credentials, contacts, messages, calendar events, LinkedIn-derived records, retrieval records, scores, briefs, dossiers, drafts, memory records, files, and generated artifacts where technically feasible.
- ☐ Implement a deletion or disconnect flow appropriate for the beta product.
- ☐ Add a verification step confirming that user-owned records are removed or disconnected according to the documented behavior.

- ☐ Document any limitations, such as non-sensitive aggregate cost records that may remain for operational accounting.
- ☐ Test that deleting or disconnecting one user does not remove or alter another user's data.

Task 5.6: Prepare internal operations runbook

- ☐ Document what to do when Gmail ingestion fails, Calendar ingestion fails, LinkedIn-derived import fails, model calls fail, generated output quality drops, costs spike, or latency increases.
- ☐ Document the steps for refreshing account access, rerunning ingestion, reviewing logs, disabling a failing workflow, and escalating unresolved issues.
- ☐ Document known limitations and expected beta failure modes so the team can respond consistently.

Task 5.7: Prepare user-facing documentation and onboarding flow

- ☐ Write a short onboarding guide explaining supported data sources, setup steps, what ANDI generates, what it does not do, and how to provide feedback.
- ☐ Explain that generated drafts and recommendations require user review and approval before any outreach is sent.
- ☐ Document known limitations for version one, including data-source limitations, model-output limitations, and expected beta rough edges.
- ☐ Create a feedback form, notes template, or review workflow for beta users to mark useful outputs, false positives, missing context, and confusing recommendations.

Task 5.8: Build or finalize the minimum user-facing surface

- ☐ Confirm whether the beta surface is a lightweight web interface, internal dashboard, or integration-style surface.
- ☐ Ensure a non-technical beta user can review the minimum flow without developer assistance once account access is configured.
- ☐ Provide access to contact ranking, morning brief or equivalent summary, dossier view, and draft/recommendation review.
- ☐ Include source references or explanation notes where possible so users can understand why ANDI produced an output.
- ☐ Make feedback capture available from the surface or through a clearly documented companion process.

Task 5.9: Run controlled beta validation

- ☐ Onboard approved beta user(s) only after access, privacy, and monitoring checks are complete.
- ☐ Run the minimum beta workflow: connect/import data, process records, generate ranking, generate brief or summary, generate dossier, review draft/recommendation, and collect feedback.
- ☐ Record useful outputs, false positives, missing context, accepted/rejected recommendations, major errors, total cost, token usage, and average latency where available.
- ☐ If external beta onboarding is deferred, complete the same validation with an internal test user and clearly state that external beta usage has not yet occurred.

Detailed Acceptance Criteria

- **Beta deployment readiness:** ANDI must run in the agreed hosted beta environment, not only on a developer laptop. The team must be able to provide a deployment verification record, internal access path, or walkthrough showing the system running in the beta environment.
- **Minimum beta workflow:** A user must be able to complete or observe the core version-one flow: connect/import approved data, process contacts and relationship history, review contact ranking, review a morning brief or equivalent summary, inspect a dossier, review a draft or recommendation, and provide feedback.

- **Version-one data-source boundary:** The beta product must stay within Gmail, Calendar, and LinkedIn-derived relationship data unless an additional source is separately approved. Additional integrations are not assumed as part of this milestone.
- **Per-user data separation:** The team must provide test evidence showing that one user's contacts, source records, generated outputs, preferences, and account configuration do not appear in another user's workflow.
- **Shared-contact safety:** If two users have the same contact or organization in their data, ANDI must not merge private relationship history across users unless that behavior is explicitly designed and approved.
- **Cost and token visibility:** The team must be able to report cost per user, cost per brief, cost per dossier, cost per draft, and token usage by workflow for the beta environment.
- **Reliability visibility:** The team must be able to review latency, error rate, ingestion status, account-access issues, retrieval issues, and model-call failures through a dashboard, report, or exported log summary.
- **Deletion behavior:** The user-data deletion or disconnect flow must be implemented, documented, and tested. Test evidence must show what data is removed or disconnected and what, if anything, remains for operational reasons.
- **No automatic sending:** Generated emails, messages, recommendations, or outreach actions must not be sent automatically in version one. Any draft or suggested action must require explicit user review and approval before sending.
- **User documentation:** The onboarding guide must explain supported data sources, setup steps, expected outputs, review-before-send behavior, known limitations, deletion/disconnect behavior, and how to provide feedback.
- **Operations readiness:** The internal runbook must cover the most common beta support scenarios: ingestion failure, account-access failure, LinkedIn-derived import failure, model-service failure, high cost, high latency, poor output quality, and user-support escalation.
- **Beta validation evidence:** At least one approved beta user or internal test user must review meaningful ANDI output, such as a contact ranking, brief, dossier, recommendation, or draft. Merely creating an account does not count as validation.
- **Known limitations documented:** The beta-readiness report must clearly list what is working, what is still limited, what was deferred, current cost profile, unresolved risks, and recommended next steps.

Validation Evidence Required

- Hosted beta deployment verification or walkthrough record.
- Final beta data-structure notes and successful clean setup test.
- Per-user data separation test output.
- Monitoring dashboard screenshot, report, or exported metrics covering cost, token usage, latency, and error categories.
- Data deletion or disconnect test log and verification output.
- Internal operations runbook.
- User-facing onboarding guide and known-limitations note.
- Minimum product-surface walkthrough or screenshots.
- Beta validation notes for approved beta users, or internal test-user notes if external beta onboarding is deferred.
- Beta-readiness report with known limitations and post-beta recommendations.

Risk Factors

- No dedicated frontend developer is currently on the team. If a polished frontend is required, it should be separately scoped or the v1 surface should remain lightweight.
- Hosting and model costs are estimates until the Milestone 4 cost model is validated. Cost monitoring should be active before external beta use.
- Beta users may have trust concerns around generated relationship recommendations. Source references and explanation notes should be visible wherever possible.
- Data deletion can be difficult when data is spread across multiple storage systems and generated outputs. Test deletion before external beta onboarding.
- Model outputs may still miss context or produce low-value recommendations. Beta feedback should be used to identify which outputs need further tuning after v1.

Out of Scope for Milestone 5

- Additional data sources or integrations beyond Gmail, Calendar, and LinkedIn-derived relationship data unless separately approved.
- Mobile application work.
- Fully autonomous email or message sending.
- A polished consumer-grade frontend beyond what is required for beta usability.
- Advanced administration, broad compliance programs, or commercial launch requirements beyond the agreed controlled beta scope.

FINAL COMPLETION CHECKLIST

ANDI should not be considered beta-ready until the following are true:

- ☐ Ingestion works across the agreed version-one data sources and produces usable reports.
- ☐ The beta data structure supports per-user records and has been tested from a clean setup.
- ☐ Contact Knowledge Graph exists and can be inspected.
- ☐ Scoring system is calibrated against reviewed contacts.
- ☐ Morning brief or equivalent summary produces useful, low-noise signals.
- ☐ Evaluation harness runs and stores output.
- ☐ Cost per contact, cost per brief, cost per user, and token usage are documented.
- ☐ The private integration layer is separated from private scoring, ranking, dossier-generation, and user-data logic.
- ☐ Privacy and access review is complete before any external beta use of real user data.
- ☐ Generated drafts require explicit user approval before sending.
- ☐ Dossier and draft outputs include source support where possible.
- ☐ User-facing beta workflow is usable by an approved client or internal test user.
- ☐ Data deletion or disconnect behavior is documented and tested.
- ☐ Runbook, onboarding guide, known limitations, and beta-readiness report are complete.